

1 - Networking Basics

Name of report: INR_NetEng_LAB_1_Nikita_Niakhai

Task 1 - Tools

1. Install the needed dependencies for GNS3: QEMU/KVM, Docker and Wireshark.

Hint: Check that virtualization is enabled in your bios. Make sure that the user belongs to all the necessary groups after installing the tools.

I found out that guest machine (Machine inside Virtual Box) does not support virtualization, so I decided to do it on my host machine Windows and I installed all tools here.

Firstly, I must approve that my system provides virtualization functionality. I open task bar manager → Processor and there looked up for Virtualization value

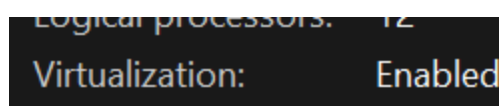


Figure 1. Task Bar Manager: Virtualization is enabled for my laptop's processor.

Then I do install all-in-one-GNS3 that additionally deploy all necessary tools such as wireguard and docker..

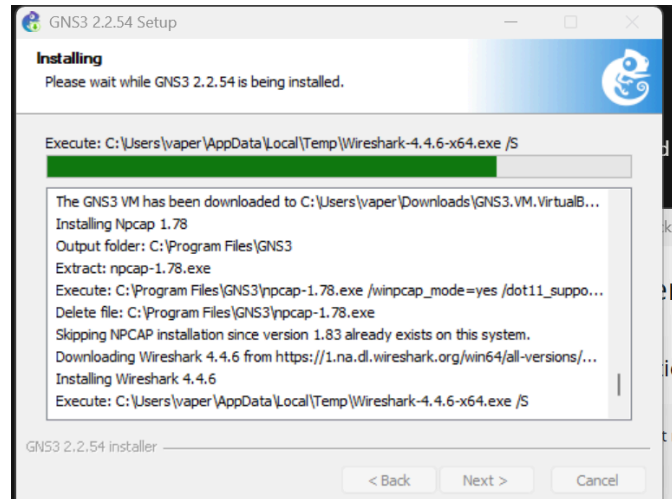


Figure 2. Installation process of GNS3 and dependencies.

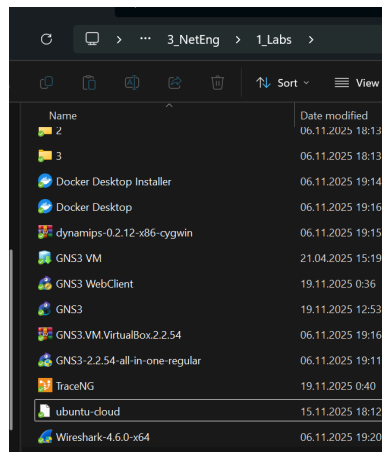


Figure 3. Tools installed for the lab, downloaded installation exe and etc inside my lab folder.

2. Start a GNS3 project, configure the pre-installed Ubuntu Cloud Guest template. Check that you can start it.

Note, that GNS3 also installed image for Virtual machine. After many unsuccessful attempts and multiple errors during activation of GNS3 VM machine inside GNS3

interface I decided to follow recommendation and installed VMware Workstation Pro.

Also I have installed Ubuntu Cloud Guest template `.ova` file [`ubuntu-cloud`], check Figure 3.

I have configured adapters of VM for proper value to use HOST-ONLY and NAT provided by VMnet.

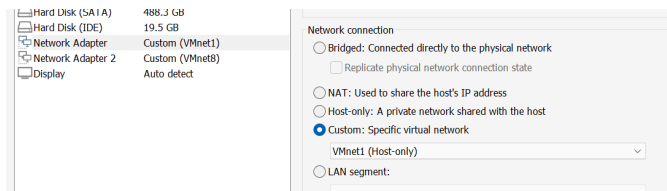


Figure 4. Network settings for GNS3 VM inside VMware.

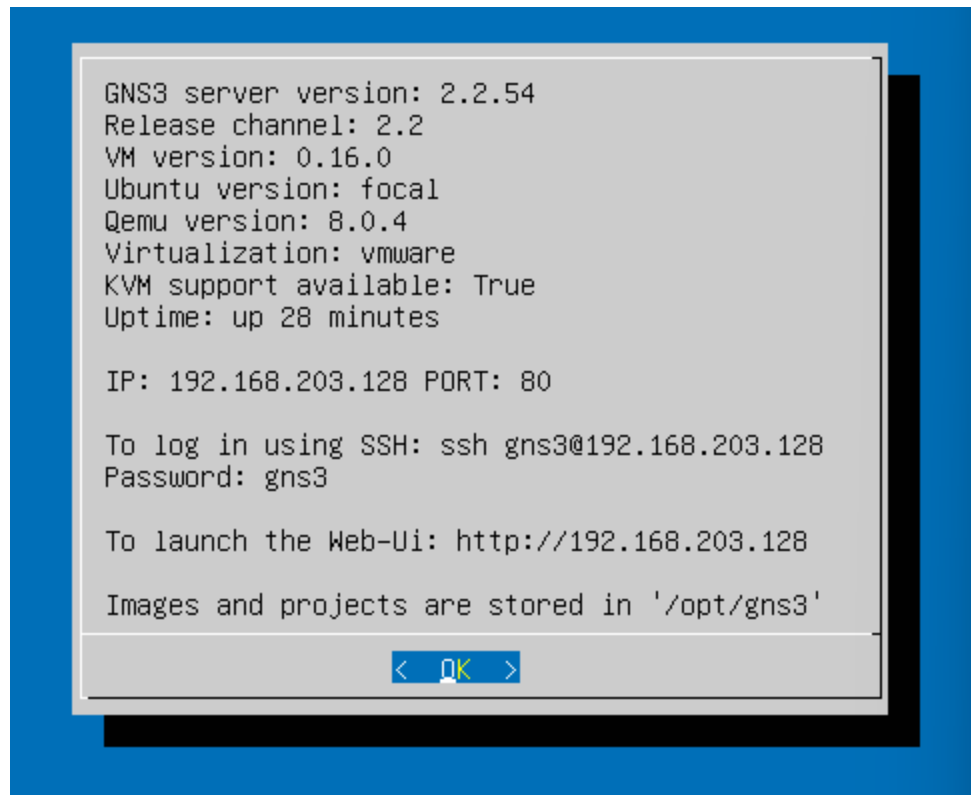


Figure 5. GNS3VM GUI is running.

For proper work of template I must check needed packages and modules on my VM, listed bellow:

```
gns3@gns3vm:~$ gns3server --version
2.2.54
```

Figure 6. Version of `gns3-server` [1]

```
gns3@gns3vm:~$ wireshark --version
Wireshark 3.2.3 (Git v3.2.3 packaged as 3.2.3-1)
```

Figure 7. Version of `wireshark`

```
gns3@gns3vm:~$ docker --version
Docker version 28.1.1, build 4eba377
```

Figure 8. Version of `docker`

```
gtk initialization failed
gns3@gns3vm:~$ kvm --version
QEMU emulator version 8.0.4 (Debian 1:8.0.4+dfsg-1ubuntu3.23.10.5~backport20.04.202407120432~ubuntu20.04.1)
Copyright (c) 2003-2022 Fabrice Bellard and the QEMU Project developers
gns3@gns3vm:~$ qemu-system-x86_64 --version
qemu-system-x86_64: --version: invalid option
gns3@gns3vm:~$ qemu-system-x86_64 --version
QEMU emulator version 8.0.4 (Debian 1:8.0.4+dfsg-1ubuntu3.23.10.5~backport20.04.202407120432~ubuntu20.04.1)
Copyright (c) 2003-2022 Fabrice Bellard and the QEMU Project developers
```

Figure 9. Versions of `kvm` and `qemu`

Now when everything is running and setup, I inspect Edit→Preferences→GNS3 VM. VM rules are setup correctly.

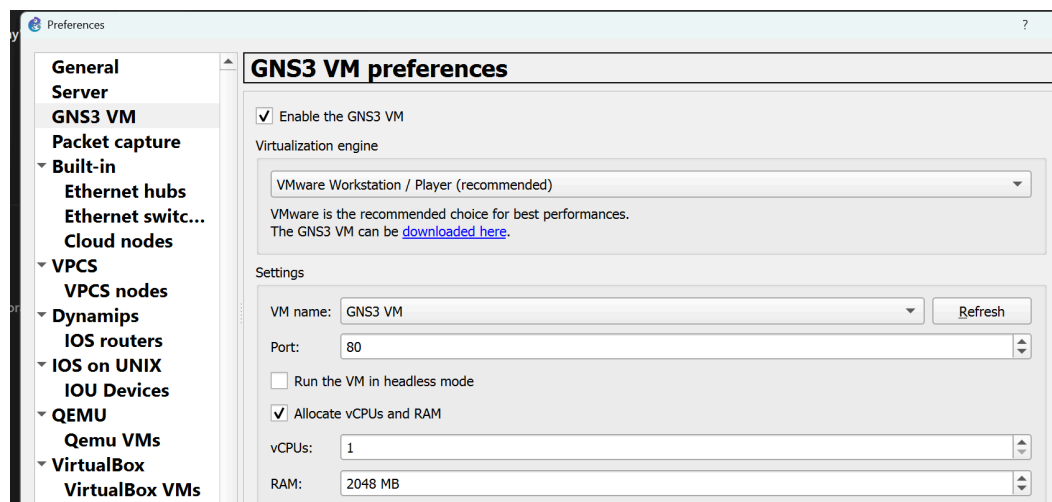


Figure 10. VM rules

Servers and nodes are active and we can observe it on Figure 11

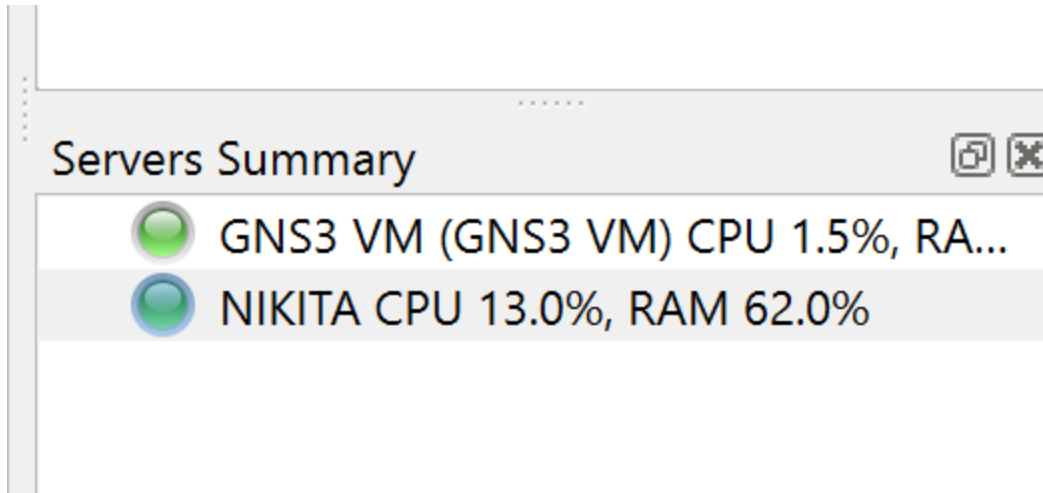


Figure 11. Servers summary

Therefore, I will install Ubuntu Cloud Guest appliance on my VM (File → Import Appliance → Select my ova), look at figure 12. There I downloaded build files from Jammy Jellyfish and now they are available to be executed.

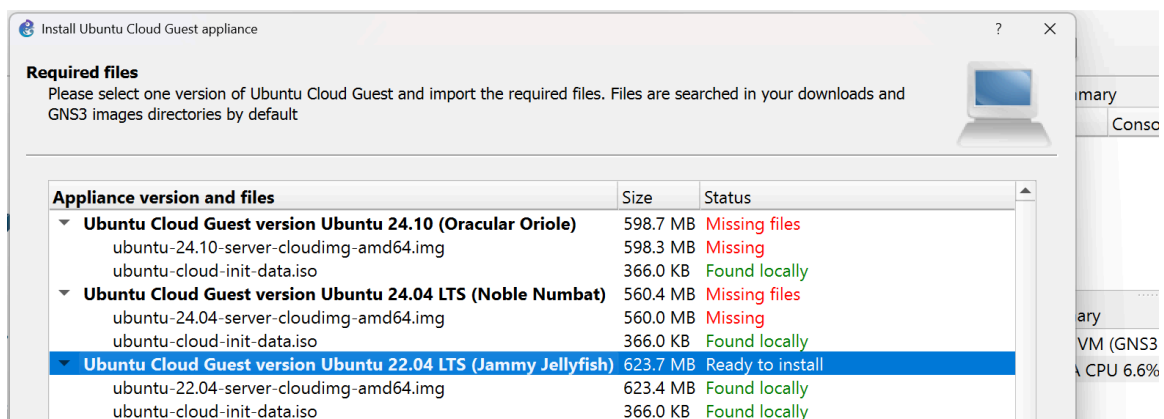


Figure 12. Installation of Ubuntu Cloud Guest

Results of installation are seen in figure 13.

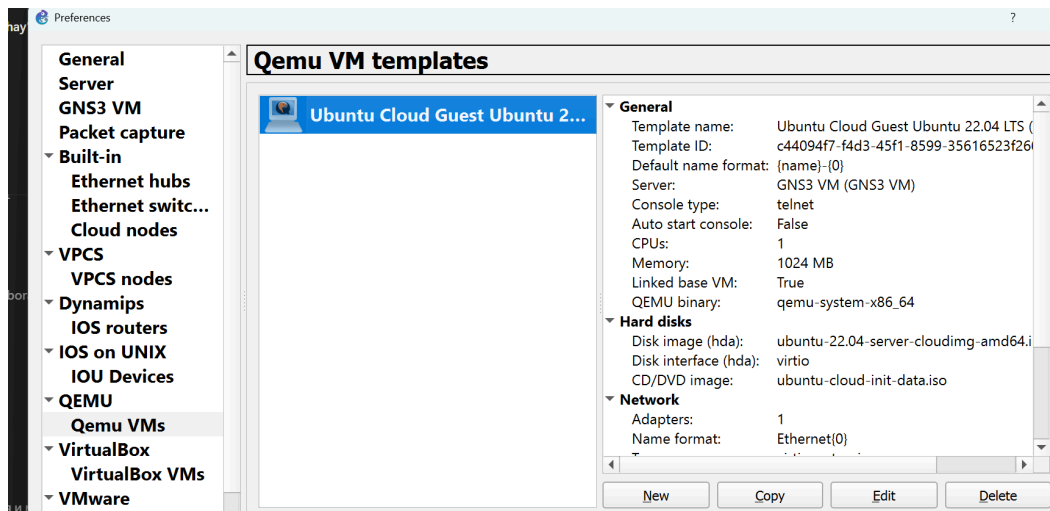


Figure 13. Rules for QEMU VM templates inside GNS3 Preferences (Ubuntu Cloud Guest is installed)

3. What are the different ways you can configure internet access in GNS3?
 Test them with a single created VM and give a one-line description of each.
 What are the differences between them?
 Bonus: show the difference between them on practice and test the connectivity.

- **NAT NODE (Node → NAT)**

traffic is flying only inside private network and any traffic e.g. from outside internet can't reach the private network.

NAT - Network Address Translation, router service that is placed on the edge and is there to connect private network to public ones.

Why to use?

- easy to configure
- inbound traffic is the one allowed, outbound traffic is blocked

- **Cloud/Bridged**

VM is on real LAN, that we use on host machine. Depend on physical network. Inbound and Outbound traffic works

- **2 adapters: Host-Only + NAT (default one)**

secure (isolated), good for lab traffic, reliable, but 2 adapters and configuration are needed.

Task 2 - Switching

1. Make the network topology.

Note:



Connection to VM is not working while using VPN :(

I created project inside default folder. Then started dragging and dropping elements of first topology to my blanket. All elements will be setup as children of GNS3 VM.

I created nodes from my Ubuntu Cloud configured on VM. Then renamed them, started, accessed via Console.

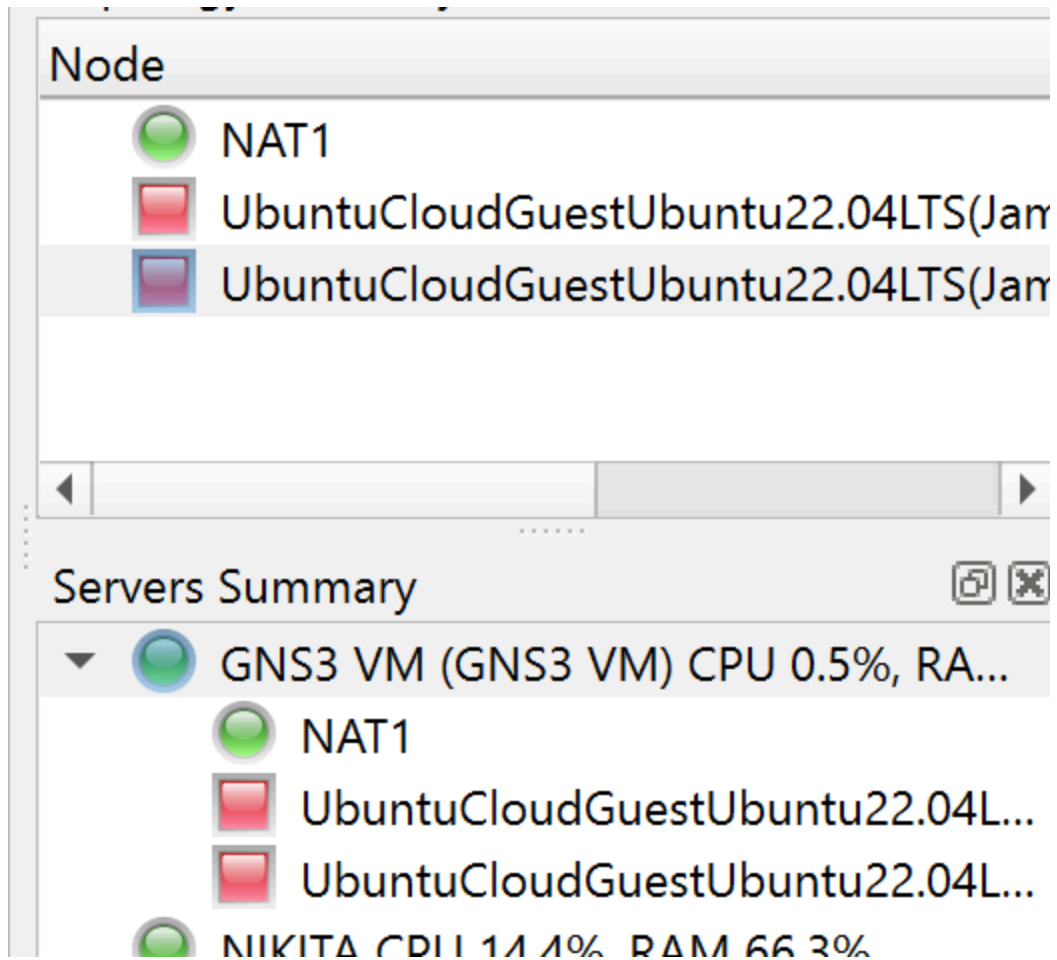


Figure 14. Servers Summary for nodes

I setup NAT for GNS3 VM server.

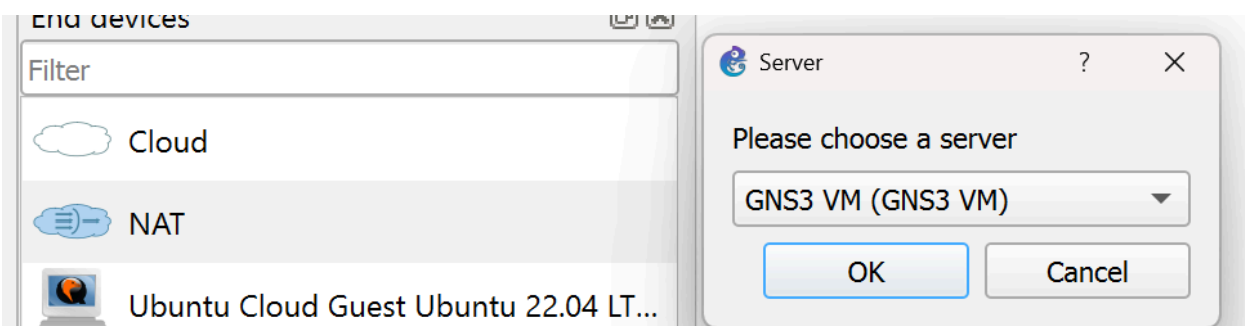


Figure 15. Generation of NAT

I used **cable** to connect elements between each other, choosed proper ports and outlined that with notes on blanket. The whole topology is seen in figure 16.

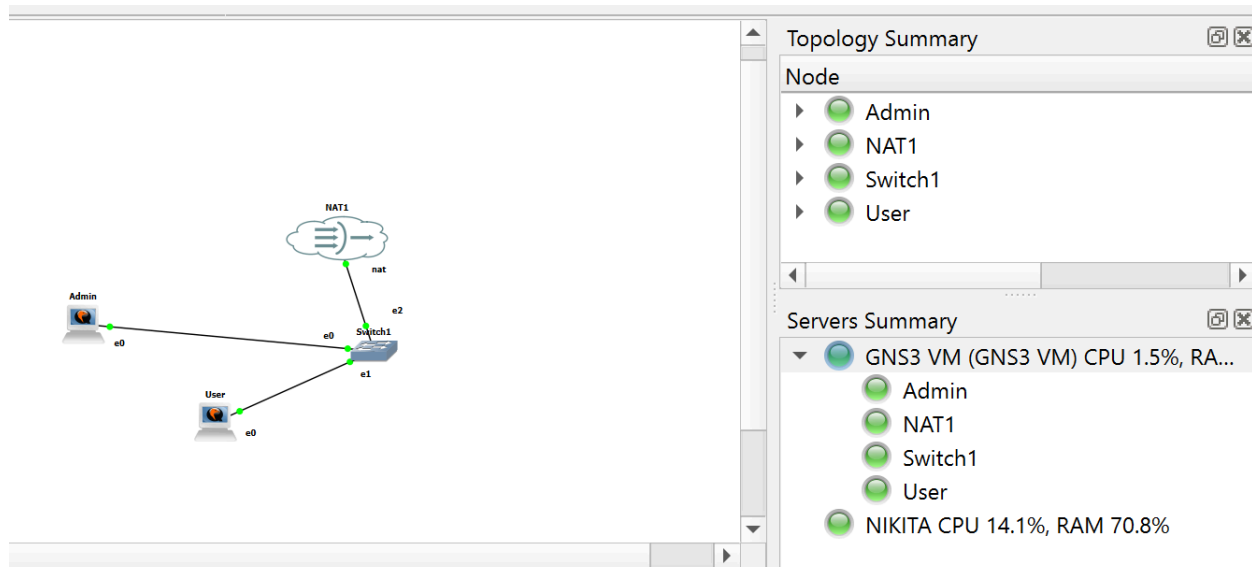


Figure 16. Topology

When I started all nodes console with booting process appeared, after the process finished, I run console for each node.

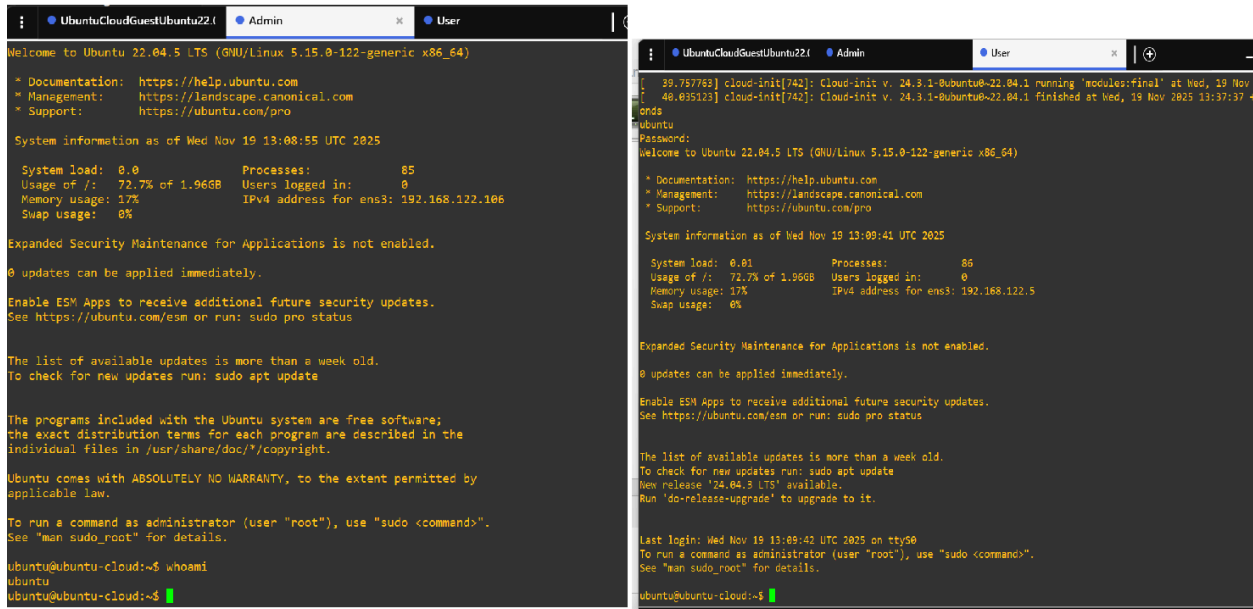


Figure 17. Console for each VM (Admin(Web-Server) and User(Admin))

2. Install openssh-server on both VMs and nginx web server on the Web VM.

`ssh` is already configured on VM, so I just verify that process is active on both machines.

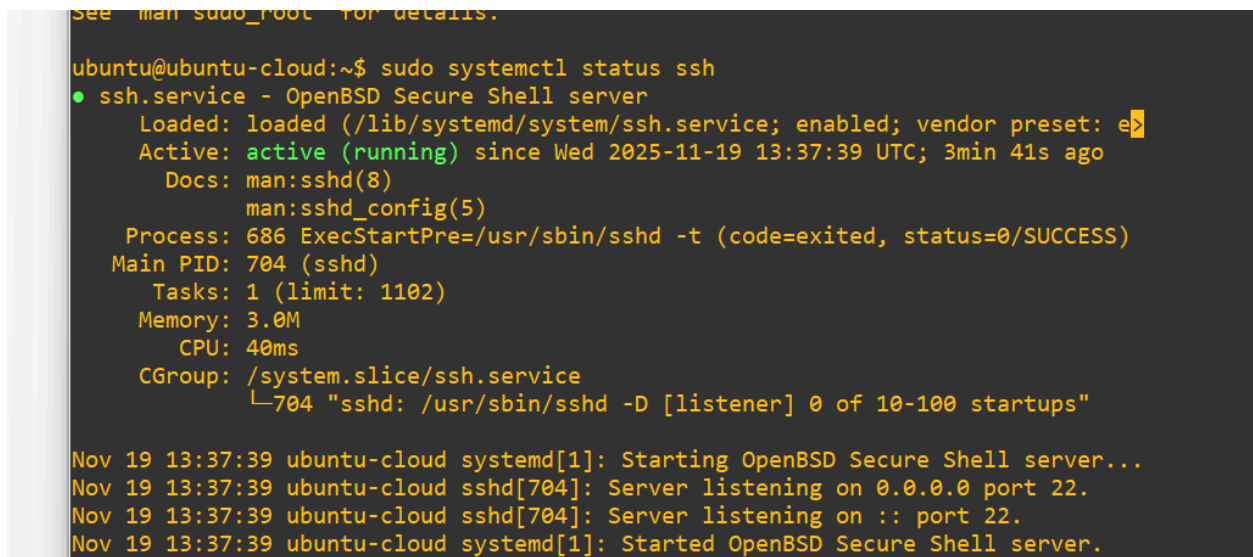


Figure 18. ssh process on web-server/admin machine.

I downloaded nginx and the needed packages on both machines.

```
0 upgraded, 0 newly installed, 0 to remove and 163 not upgraded.
ubuntu@ubuntu-cloud:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset:
   Active: active (running) since Wed 2025-11-19 13:47:54 UTC; 1min 59s ago
     Docs: man:nginx(8)
   Process: 1769 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_proce
   Process: 1770 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (c
 Main PID: 1856 (nginx)
    Tasks: 2 (limit: 1102)
   Memory: 4.1M
      CPU: 174ms
   CGroup: /system.slice/nginx.service
           └─1856 "nginx: master process /usr/sbin/nginx -g daemon on; master
              1859 "nginx: worker process" "" "" "" "" "" "" "" "" "" "" "" ""
Nov 19 13:47:53 ubuntu-cloud systemd[1]: Starting A high performance web server
Nov 19 13:47:54 ubuntu-cloud systemd[1]: Started A high performance web server
lines 1-16/16 (END)
```

Figure 19. nginx process on web-server machine.

2. What is the IP of the mask corresponding to /28 ?

How many machines can you configure under this subnet? Explain it.

So IPV4 is 32 bits. Mask is 28. $32 - 28 = 4$ bits for hosts.

Available addresses are 2^n bits for hosts $\rightarrow 2^4 = 16$

But! 2 addresses are reserved, one for server **0**, one for accessing all hosts in the subnet **1**.

14 addresses.

What is IP?

11111111.11111111.11111111.11110000

255.255.255.x?

$(1+2+4+8)+(16+32+64+128)$ - first group is 0, so decimal value for last is
 $16+32+64+128=240$.

x=240

IP for subnet is 255.255.255.240

Answer: IP for /28 mask is 255.255.255.240 (decimal),
11111111.11111111.11111111.11110000 (binary) and 14 machines are available to be
configured on the subnet.

Number for usable hosts is 14. Because if we transform mask to binary we get:

11111111.11111111.11111111.11110000

This leaves us with 4 bits available for host addresses. It gives us possibility of
2

$2^4=16$ different addresses, but first and last addresses are reserved for network
address and broadcast address respectively.

1. Configure the VMs with private static IPs under a /28 subnet.

First I did for Web server.

`ip a` showed me current IP address that I will be using for configuration of
network.

```

ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:22:d3:7a:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.122.106/24 metric 100 brd 192.168.122.255 scope global dynamic ens3
        valid_lft 2599sec preferred_lft 2599sec
    inet6 fe80::e22:d3ff:fe7a:0/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-cloud:~$ sudo nano /etc/netplan/50-cloud-init.yaml

```

Figure 20. IP address of WEB-SERVER VM before configuration changes

I modified configuration file: added `addresses` prop with value of `current_IP_of_machine/28`, look in figure

```

network:

  ethernets:
    ens3:
      addresses: [192.168.122.106/28]
      dhcp4: true
      dhcp6: true
      match:
        macaddress: 0c:22:d3:7a:00:00
        set-name: ens3
      version: 2
File Name to Write: /etc/netplan/50-cloud-init.yaml
ubuntu@ubuntu-cloud:~$ sudo netplan apply

```

Figure 21. Configuration file for network ip.

Changed config file and added addresses.

```

File Name to Write: /etc/netplan/50-cloud-init.yaml
ubuntu@ubuntu-cloud:~$ sudo netplan apply
WARNING:root:Cannot call Open vSwitch: ovssdb-server.service is not running.
ubuntu@ubuntu-cloud:~$ sudo nano /etc/netplan/50-cloud-init.yaml ^C
ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:22:d3:7a:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.122.106/28 brd 192.168.122.111 scope global ens3
        valid_lft forever preferred_lft forever
    inet 192.168.122.106/24 metric 100 brd 192.168.122.255 scope global dynamic ens3
        valid_lft 2988sec preferred_lft 2988sec
    inet6 fe80::c2:d3:7a:00:00/64 scope link
        valid_lft forever preferred_lft forever

```

Figure 22. IP address of WEB-SERVER VM after configuration changes.

At present, Admin VM to be done, the same steps will be performed. Figures 23-25

```

ubuntu@ubuntu-cloud:~$ sudo systemctl stop nginx
ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:e0:c7:d6:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.122.5/24 metric 100 brd 192.168.122.255 scope global dynamic ens3
        valid_lft 2066sec preferred_lft 2066sec
    inet6 fe80::e0:c7:ff:fe:d6:00/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-cloud:~$

```

Figure 23. IP address of Admin VM before configuration changes

I modified configuration file: added `addresses` prop with value of `current_IP_of_machine/28`

```
ubuntu@ubuntu-cloud:~$ sudo netplan apply
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service
ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue stat
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
    link/ether 0c:e0:c7:d6:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.122.5/28 brd 192.168.122.15 scope global
        valid_lft forever preferred_lft forever
    inet 192.168.122.5/24 metric 100 brd 192.168.122.255 s
        valid_lft 3600sec preferred_lft 3600sec
    inet6 fe80::ee0:c7ff:fed6:0/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-cloud:~$
```

Figure 24. IP address of Admin VM after configuration changes

```
University / ... / NetEng / 1 - Networking Basics
User x UbuntuCloudGuestUbuntu22.04 Admin
GNU nano 6.2 /etc/netplan/50-cloud-init.yaml
# This file is generated from information provided by the datasource. Changes
# to it will not persist across an instance reboot. To disable cloud-init's
# network configuration capabilities, write a file
# /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg with the following:
# network: {config: disabled}
network:
  ethernets:
    ens3:
      addresses: [192.168.122.5/28]
      dhcp4: true
      dhcp6: true
      match:
        macaddress: 0c:e0:c7:d6:00:00
      set-name: ens3
  version: 2

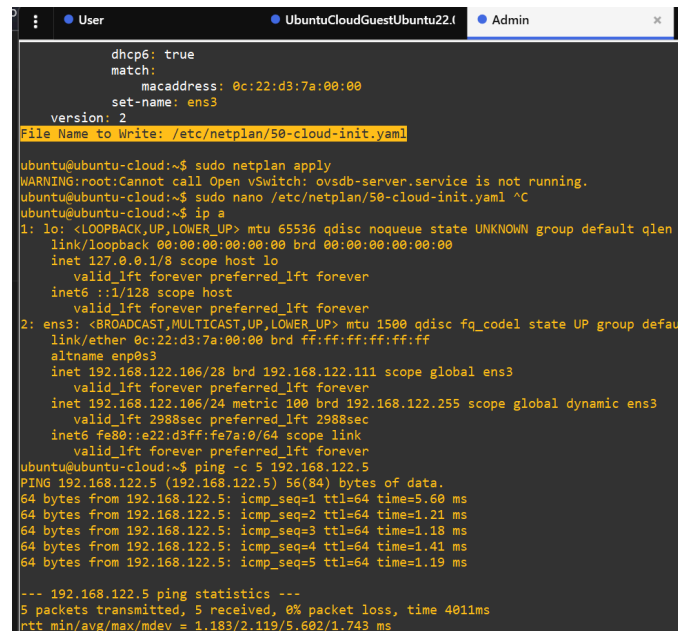
^G Help      ^O Write Out ^K Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```


Figure 25. Configuration file for Admin VM

4. Check that you have connectivity between them.

Hint: use ping, traceroute and mtr.

Connectivity is successful from both sides, figure 26.



```
dhcpc6: true
match:
  macaddress: 0c:22:d3:7a:00:00
  set-name: ens3
version: 2
File Name to Write: /etc/netplan/50-cloud-init.yaml

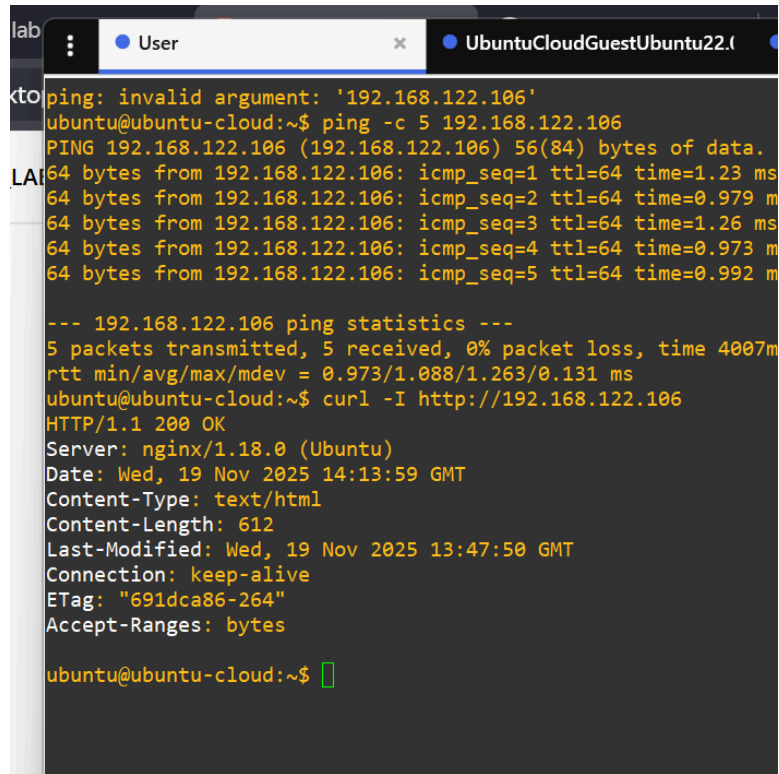
ubuntu@ubuntu-cloud:~$ sudo netplan apply
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.
ubuntu@ubuntu-cloud:~$ sudo nano /etc/netplan/50-cloud-init.yaml ^C
ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
    link/ether 0c:22:d3:7a:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.122.106/28 brd 192.168.122.111 scope global ens3
        valid_lft forever preferred_lft forever
    inet 192.168.122.106/24 metric 100 brd 192.168.122.255 scope global dynamic ens3
        valid_lft 2988sec preferred_lft 2988sec
    inet6 fe80::e22:d3ff:fe7a:0/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-cloud:~$ ping -c 5 192.168.122.5
PING 192.168.122.5 (192.168.122.5) 56(84) bytes of data:
64 bytes from 192.168.122.5: icmp_seq=1 ttl=64 time=5.60 ms
64 bytes from 192.168.122.5: icmp_seq=2 ttl=64 time=1.21 ms
64 bytes from 192.168.122.5: icmp_seq=3 ttl=64 time=1.18 ms
64 bytes from 192.168.122.5: icmp_seq=4 ttl=64 time=1.41 ms
64 bytes from 192.168.122.5: icmp_seq=5 ttl=64 time=1.19 ms

--- 192.168.122.5 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 401ms
rtt min/avg/max/mdev = 1.183/2.119/5.602/1.743 ms
```

Figure 26. **Curl** is used to check accessibility of Admin machine from Web-Server

5. Make sure your web server is accessible from the Admin VM.

Web server is accessible from the Admin VM, figure 27.

A terminal window with a dark background and yellow text. The window has two tabs: 'User' and 'UbuntuCloudGuestUbuntu22.1'. The terminal shows a sequence of commands and their outputs. First, a 'ping' command is attempted with an invalid argument, followed by a successful 'ping -c 5 192.168.122.106' command showing five successful pings with varying times. Then, a 'curl -I http://192.168.122.106' command is executed, returning an HTTP 200 OK status and various headers including 'Server: nginx/1.18.0 (Ubuntu)', 'Date: Wed, 19 Nov 2025 14:13:59 GMT', 'Content-Type: text/html', 'Content-Length: 612', 'Last-Modified: Wed, 19 Nov 2025 13:47:50 GMT', 'Connection: keep-alive', 'ETag: "691dca86-264"', and 'Accept-Ranges: bytes'. The prompt 'ubuntu@ubuntu-cloud:~\$' is visible at the bottom.

```
ping: invalid argument: '192.168.122.106'
ubuntu@ubuntu-cloud:~$ ping -c 5 192.168.122.106
PING 192.168.122.106 (192.168.122.106) 56(84) bytes of data.
64 bytes from 192.168.122.106: icmp_seq=1 ttl=64 time=1.23 ms
64 bytes from 192.168.122.106: icmp_seq=2 ttl=64 time=0.979 ms
64 bytes from 192.168.122.106: icmp_seq=3 ttl=64 time=1.26 ms
64 bytes from 192.168.122.106: icmp_seq=4 ttl=64 time=0.973 ms
64 bytes from 192.168.122.106: icmp_seq=5 ttl=64 time=0.992 ms

--- 192.168.122.106 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 0.973/1.088/1.263/0.131 ms
ubuntu@ubuntu-cloud:~$ curl -I http://192.168.122.106
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 19 Nov 2025 14:13:59 GMT
Content-Type: text/html
Content-Length: 612
Last-Modified: Wed, 19 Nov 2025 13:47:50 GMT
Connection: keep-alive
ETag: "691dca86-264"
Accept-Ranges: bytes

ubuntu@ubuntu-cloud:~$
```

Figure 27. Check accessibility of web-server from user using `curl` command.

Task 3 - Routing

1. Select a virtual Routing solution (Gateway) such as Mikrotik (recommended default choice), PfSense, VyOS, Untangle NG, OpenWrt, Cumulus VX.
2. Create Internal network for Worker instance.
3. Connect your Gateway to the internet and to your workstation/host.
4. Setup the gateway for Admin, Web and Worker, then check their connectivity.
5. Configure port forwarding for http and ssh to Web and Admin respectively.
6. Check that you can ssh to the Admin and access your web page from your workstation/host.

Application / References

- [1] Documentation for GNS3 : for installing gns3 on VM :
<https://docs.gns3.com/docs/getting-started/installation/linux/>
 - [2] Official site of GNS3 : for installing distribution
 - [3] Official site of VMware and their provider : to install VMware Workstation
 - [4] GPT for assistance in practical part and explaining theory.
-

Previous attempts on Task 1

```
(nikita@nikita)-[~]  
$ sudo apt install -y qemu-kvm libvirt-d  
docker.io wireshark  
Note, selecting 'qemu-system-x86' instead  
Upgrading:  
    docker-cli                libgtksourceview-4
```

```
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils virt-manager docker.io wireshark
```

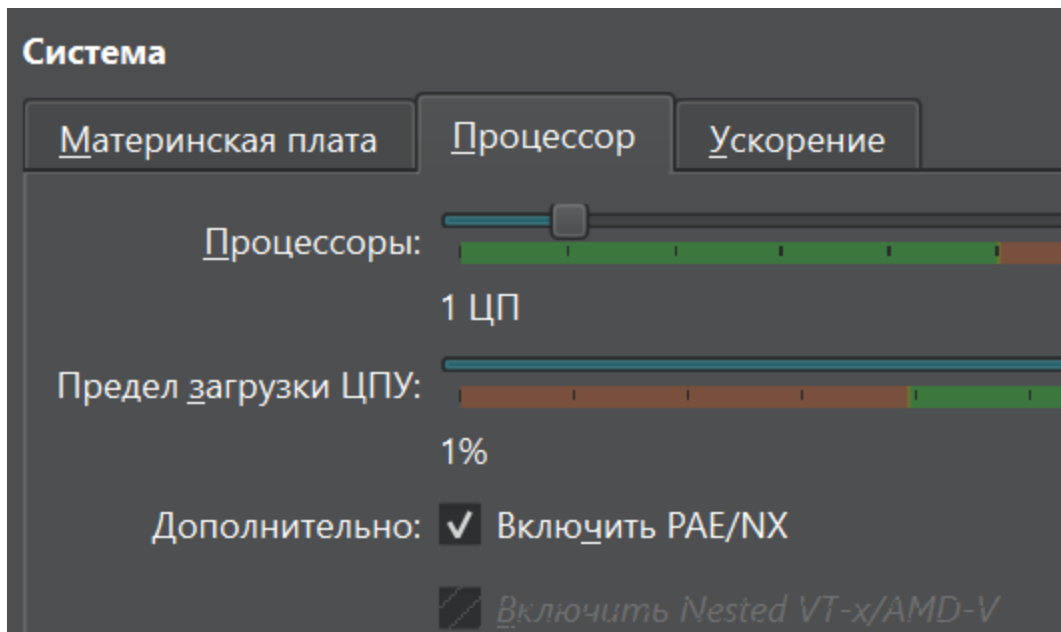
```
(nikita@nikita)-[~]  
$ sudo apt install -y qemu-system-x86  
qemu-system-x86 is already the newest vers  
ion (1:10.1.2+ds-1).  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0  
  , Not Upgrading: 1372
```

```
sudo apt install -y qemu-system-x86
```

Then I do restart of the machine

```
, Not upgrading: 13/2  
  
(nikita@nikita)-[~]  
$ egrep -c '(vmx|svm)' /proc/cpuinfo  
0  
  
(nikita@nikita)-[~]
```

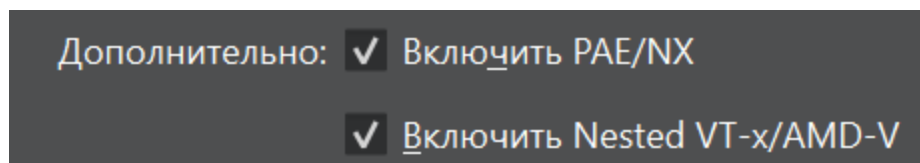
Check if "virtualization is enabled in your bios" `egrep -c '(vmx|svm)' /proc/cpuinfo` : I need to enable it.



Now in the settings I don't have this option available. To enable it I should run the command inside Windows (my normal machine) folder:

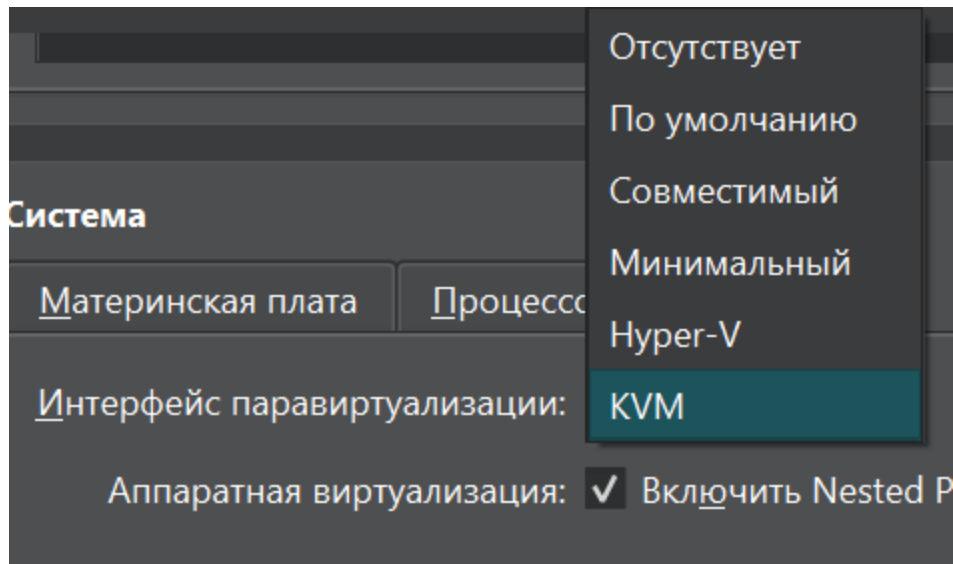
```
VBoxManage modifyvm "Dev Machine 2" --nested-hw-virt on
```

```
C:\Program Files\Oracle\VirtualBox>VBoxManage modifyvm "Dev Machine 2" --nested-hw-virt on
```



```
C:\Program Files\Oracle\VirtualBox>VBoxManage showvminfo "Dev Machine 2" | find "Nested"  
Nested VT-x/AMD-V:      enabled  
Nested Paging:         enabled
```

Now it is enabled.



Now I choose KVM as Paravirtualization interface.

I boot machine again. And I still need to do things to my host machine such as

```
bcdedit /set hypervisorlaunchtype off
```

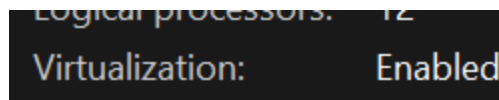
```
dism.exe /Online /Disable-Feature:Microsoft-Hyper-V
```

```
dism.exe /Online /Disable-Feature:VirtualMachinePlatform
```

```
dism.exe /Online /Disable-Feature:WindowsHypervisorPlatform
```

```
dism.exe /Online /Disable-Feature:Containers
```

Now I found out that guest machine does not support virtualization, so I decided to do it on my host machine Windows and I installed all tools.



Virt. is enabled for processor.

